

Python Fundamentals (Module-2)

Character set

A character set is a set of valid characters acceptable by a programming language in scripting. In this case, we are talking about the Python programming language. So, the Python character set is a valid set of characters recognized by the Python language. These are the characters we can use during writing a script in Python. Python supports all ASCII / Unicode characters that include:

- **Alphabets:** All capital (A-Z) and small (a-z) alphabets.
- **Digits:** All digits 0-9.
- **Special Symbols:** Python supports all kind of special symbols like, " ' | ; : ! ~ @ # \$ % ^ ` & * () _ + - = { } [] \ .
- **White Spaces:** White spaces like tab space, blank space, newline, and carriage return.
- **Other:** All ASCII and UNICODE characters are supported by Python that constitutes the Python character set.

Tokens

A token is the smallest individual unit in a python program. All statements and instructions in a program are built with tokens. The various tokens in python are:

1. Keywords: Keywords are words that have some special meaning or significance in a programming language. They can't be used as variable names, function names, or any other random purpose. They are used for their special features. In Python we have 33 keywords some of them are: try, False, True, class, break, continue, and, as, assert, while, for, in, raise, except, or, not, if, elif, print, import, etc.

2. Identifiers: The names assigned to any variable, class, function, list, method, etc., for their recognition are known as identifiers. Python has certain guidelines for naming identifiers and is a case-sensitive programming language. To create an identifier, follow these guidelines: -

- Python is case-sensitive, as was already mentioned. So, when defining identifiers, the case is important. Thus, the terms "python" and "Python" are not interchangeable.
- A capital alphabet (A-Z), a small alphabet (a-z), or an underscore (_) precedes the word "identifier." No other character can be used as the starting point.
- Digits are also allowed to be a member of the identifier, along with the letters and underscores, but they cannot be the initial character.
- Any additional special symbols or whitespace must be avoided when creating an identity.
- A keyword cannot be an identifier.

3. Literals: The constant values or data objects utilized in a source program are known as literals. Various types of literal are:

String Literals: The Python string literals are represented by text enclosed in solo, double, or triple quotations. Examples include "Computer Programming," "sigma," etc. Multi-line phrases can also be written using triple quotations.

Integer Literal: Contains 0 as well as positive and negative values. There are no fractional pieces included. In addition, literals in binary, decimal, octal, and hexadecimal are permitted. E.g. 23, -56 etc.

Float Literal: It contains real numbers in the positive and negative ranges. Fractional portions are also a part of it. E.g. 23.8, -67.5 etc.

Complex Literal: This contains the numerals a+bj, where a stands for the fundamental component and b for the complex component. E.g. 2+3j

Boolean Literals: In Python, Boolean tokens only have two possible values either True or False.

Special Literals: The special literal "None" in Python exists. It is employed to signify emptiness, or nothing.

Literals Collections

List, dictionary, tuple, and sets are examples of Python literal collections.

- **List:** It is a comma-separated list of components enclosed in square brackets. These elements can be of any data type and can be changed.
- **Tuple:** In round brackets, this is similar to a list having comma-separated items or values. The values are fixed and can have any Python data type.
- **Dictionary:** This data structure is an unordered set of key-value combinations.
- **Set:** It is the group of components enclosed in curly braces, "{}"

4. Operators: These are the tokens responsible to perform an operation in an expression. The variables on which operation is applied are called operands. Operators can be unary or binary. Unary operators are the ones acting on a single operand like complement operator, etc. While binary operators need two operands to operate. E.g. +, -, * etc.

5. Punctuators: These are the symbols that used in Python to organize the structures, statements, and expressions. Some of the Punctuators are: [] { } () @ -= += *= /= **== = , etc.

Printing in Python

Single Line -> print('hello')
Single Line -> print("hello")
Multi Line -> print("""hello
how""")

Comments in Python

Single Line -> #
Multi Line -> ''' ''' (Triple Single quote) or """ """ (Triple double quote)

Indentation in Python

The first line of python code cannot have Indentation. Indentation is mandatory in python to define the blocks of statements. The number of spaces must be uniform in a block of code. It is preferred to use whitespaces instead of tabs to indent in python.

E.g.,

```
if a>b:
```

```
    print(a)
```

```
print(b)
```

Here 'print (a)' is under the body of if statement but not 'print (b)'.

Python Variables

Variables are containers for storing data values. Python has no command for declaring a variable. A variable is created the moment you first assign a value to it.

E.g.,

```
x=5
```

```
y="Hello"
```

Get the Type

You can get the data type of a variable with the `type()` function.

```
x=5  
print(type(x))
```

Output

```
<class 'int'>
```

Get the id

You can get the identity of a variable with the `id()` function. If 2 variables contain same value, the id for both the variables will be same.

```
x=5  
y=5  
print(id(x), id(y))
```

Output

```
1263527 1263527 (it may differ in the time of execution)
```

Naming a Variable

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

Many Values to Multiple Variables

```
x, y, z = "Orange", "Banana", "Cherry"  
print(x)  
print(y)  
print(z)
```

One Value to Multiple Variables

```
x = y = z = "Orange"  
print(x)  
print(y)  
print(z)
```

Unpack a Collection

If you have a collection of values in a list, tuple etc. Python allows you to extract the values into variables. This is called *unpacking*

```
fruits = ["apple", "banana", "cherry"]  
x, y, z = fruits  
print(x,y,z) The output will be apple banana cherry
```